

Functional Programming

1. Final data structures

In Clojure, data structures like maps and vectors are immutable by default. This means you can't accidentally change your data – any change creates a new version.

Example:

```
1 (def destination {:city "Paris" :country "France" :weather "cold"})
2
3 (def updated-destination (assoc destination :weather "warm"))
4
5 (println destination)
6 (println updated-destination)
7 |
```

Output:

Output	Input	Comments
0		
{:city Paris, :country France, :weather cold}		
{:city Paris, :country France, :weather warm}		

I found this really useful because I didn't have to worry about accidentally breaking something when updating data.

2. Side-effect-free functions

A pure function takes inputs, does something with them, and returns a result – no side effects like changing global data.

Example: Check if a destination matches a user's preferred weather.

```

1 (defn matches-preferences? [destination weather]
2   (= (:weather destination) weather))
3
4 (def paris {:city "Paris" :weather "cold"})
5
6 (println (matches-preferences? paris "cold"))
7 (println (matches-preferences? paris "warm"))
8
9

```

Output:

Output	Input	Comments
true		
false		

This function is simple. It just compares values and gives an answer. It doesn't mess with anything else, which makes it reliable.

3. Higher-order functions

Higher-order functions are functions that take other functions as arguments or return new functions. In Clojure, `filter` and `map` are perfect examples.

Example: Filter destinations based on weather.

```

1 (def destinations [{:city "Paris" :weather "cold"}
2                   {:city "Rome" :weather "warm"}
3                   {:city "Oslo" :weather "cold"}])
4
5 (defn filter-by-weather [dest-list preferred-weather]
6   (filter #(= (:weather %)) preferred-weather) dest-list))
7
8 (println (filter-by-weather destinations "cold"))
9

```

Output:

Output	Input	Comments
({:city Paris, :weather cold} {:city Oslo, :weather cold})		

4. Functions as parameters and return values

In Clojure, you can pass a function to another function or even return a function.

Example: Sorting destinations by budget.

```
1 (defn sort-destinations [dest-list key-fn]
2   (sort-by key-fn dest-list))
3
4 (def destinations [{:city "Paris" :budget 500}
5                   {:city "Rome" :budget 300}
6                   {:city "Oslo" :budget 700}])
7
8 (println (sort-destinations destinations :budget))
9
```

Output:

Output	Input	Comments
0		
({:city Rome, :budget 300} {:city Paris, :budget 500} {:city Oslo, :budget 700})		

“:budget” is passed as a “function” that tells sort-by what to sort on.

I didn’t have to hardcode the sorting logic, which makes it reusable for other keys.

5. Closures / anonymous functions

Closures are functions that remember the variables they were created with. Anonymous functions are just quick, unnamed functions for small tasks.

Example: Create a budget filter.

```
1 (defn budget-filter [max-budget]
2   (fn [destination] (<= (:budget destination) max-budget)))
3
4 (def destinations [{:city "Paris" :budget 500}
5                   {:city "Rome" :budget 300}
6                   {:city "Oslo" :budget 700}])
7
8 (def filter-by-budget (budget-filter 500))
9
10 (println (filter filter-by-budget destinations))
11
```

Output:

Output	Input	Comments
({:city Paris, :budget 500} {:city Rome, :budget 300})		0

Using anonymous functions is really helpful when you just need a small, quick function.